

Projet de groupe - Seance 4

Projet de groupe : concevez votre propre API

Séance 4 · Trinômes · 4h · Aucune nouvelle théorie aujourd'hui, place à la synthèse

Contexte

Depuis 3 séances, vous avez construit GameGuard : des routes REST, un CRUD complet, des modèles Pydantic validés en profondeur. Aujourd'hui, pas de nouveau cours. Vous allez concevoir et coder, en groupe de 3, votre propre API originale, de zéro, en réutilisant tout ce que vous avez appris, et en travaillant sur un dépôt GitHub partagé comme dans un vrai projet d'équipe.

Ce projet est volontairement long et dense. Il est construit en 5 paliers, du plus simple au plus exigeant. Personne n'est censé aller jusqu'au bout, l'objectif est d'avancer le plus loin possible en comprenant réellement ce que vous produisez.

Travailler avec l'IA

L'utilisation d'un assistant IA (ChatGPT, Claude, Copilot ou autre) est autorisée pour vous aider à avancer plus vite. Prenez cependant le temps de relire et comprendre ce que l'IA vous propose avant de l'intégrer : un code que vous ne maîtrisez pas devient très difficile à corriger ou à faire évoluer plus tard dans le projet, et une partie de la note porte justement sur la qualité de votre compréhension collective du projet, visible à travers votre journal de bord et votre présentation finale.

Organisation avec GitHub

Chaque groupe travaille sur un **dépôt GitHub dédié et public**, plutôt que sur des fichiers en local partagés par clé USB ou par mail. C'est l'occasion de pratiquer un vrai workflow d'équipe.

Mise en place, en début de séance

- Un membre du groupe crée un dépôt public sur GitHub, avec un nom clair : `projet4-nomdugroupe` (par exemple `projet4-lesbackendeurs`)
- Ce membre ajoute les deux autres comme collaborateurs (Settings puis Collaborators sur GitHub), ou plus simplement, les trois membres travaillent avec le même compte cloné en local si vous préférez rester simple
- Communiquez-moi l'URL de votre dépôt dès qu'il est créé, pendant la phase de validation du thème

Pendant la séance

- Chaque membre **clone le dépôt** sur sa machine et travaille dessus localement
- Faites des **commits réguliers**, pas un seul gros commit à la fin de la séance. Un commit à chaque fonctionnalité terminée (un modèle, une route, un validator) donne un historique lisible de la progression du projet
- Écrivez des **messages de commit clairs** : `ajout du modele Recipe avec validation`, plutôt que `update` ou `fix`
- Poussez (`git push`) régulièrement vos commits sur le dépôt partagé, pour que vos coéquipiers voient votre avancée en temps réel
- En cas de conflit Git (deux personnes ont modifié le même fichier en même temps), c'est normal, demandez de l'aide si vous ne savez pas résoudre un conflit

Pourquoi un dépôt public

- Ça facilite le test croisé de la fin de séance : l'autre groupe n'a qu'à cloner votre dépôt avec l'URL, sans transfert de fichiers compliqué
- Ça vous permet de commencer un vrai historique de projet, montrable ensuite dans un portfolio
- L'historique des commits donne une vision claire de la progression du groupe tout au long de la séance

Structure attendue du dépôt

```
projet4-nomdugroupe/
├─ main.py
├─ requirements.txt
├─ README.md
├─ JOURNAL.md
├─ TESTS.md
└─ REVIEW.md          (ajouté après le test croisé)
```

Choisir votre thème

Le thème doit être différent de GameGuard, et suffisamment riche pour contenir au moins 5 ressources liées entre elles de façon cohérente. Quelques pistes si vous manquez d’inspiration (vous n’êtes pas obligés de les utiliser) :

- Une plateforme de location de matériel (objets, catégories, réservations, utilisateurs, avis)
- Un système de gestion d’une salle de sport (adhérents, cours, coachs, réservations, équipements)
- Une plateforme de covoiturage (trajets, conducteurs, passagers, véhicules, avis)
- Un système de gestion d’une bibliothèque universitaire élargie (livres, auteurs, emprunteurs, emprunts, catégories)
- Une plateforme de recettes de cuisine (recettes, ingrédients, auteurs, catégories, avis)
- Un système de billetterie d’événements (événements, lieux, billets, participants, catégories)

Validez votre thème et l’URL de votre dépôt GitHub à l’oral avec moi avant de commencer à coder, 10 à 15 minutes maximum, pour être sûrs que le thème est assez riche pour couvrir plusieurs paliers.

Palier 1, Fondations (obligatoire)

Ce qu’on attend concrètement :

- Au moins **5 ressources distinctes**. Par exemple, pour une plateforme de recettes : `Recipe` , `Ingredient` , `Author` , `Category` , `Review`
- Au moins **2 relations logiques** différentes entre vos ressources. Par exemple : un `Review` appartient à un `Recipe` (via `recipe_id`), et un `Recipe` appartient à un `Author` (via `author_id`). Ces relations se gèrent avec un simple champ d’identifiant dans vos modèles, pas besoin de base de données aujourd’hui, vos données restent en listes Python comme depuis le début du module
- Un **CRUD complet** sur **toutes vos ressources**, pas seulement deux ou trois :
 - `POST /resource` pour créer
 - `GET /resource` pour lister
 - `GET /resource/{id}` pour le détail
 - `PATCH /resource/{id}` pour modifier
 - `DELETE /resource/{id}` pour supprimer
- Une documentation `/docs` entièrement fonctionnelle sur l’ensemble de ces routes, testée par vous-mêmes

Exemple concret (thème recettes) :

```
class Recipe(BaseModel):
    id: int
    title: str
    author_id: int
    category_id: int
    prep_time_minutes: int
```

Palier 2, Validation approfondie (obligatoire)

Ce qu’on attend concrètement, sur l’ensemble de vos modèles Pydantic, au total sur le projet :

- Au moins **3 contraintes Field** différentes (par exemple : `title: str = Field(min_length=3, max_length=100)` , `prep_time_minutes: int = Field(ge=1, le=600)`)
- Au moins **2 Enum** distincts, sur deux ressources différentes (par exemple : une difficulté de recette `EASY / MEDIUM / HARD` , un statut de réservation `PENDING / CONFIRMED / CANCELLED`)
- Au moins **2 field_validator** fonctionnels, sur deux champs différents (par exemple : interdire les caractères spéciaux dans un nom d’utilisateur)
- Au moins **2 model_validator** fonctionnels, chacun comparant au moins deux champs entre eux (par exemple : une date de fin qui ne peut pas être avant une date de début)
- Au moins un modèle avec une **liste de sous-objets imbriqués** (par exemple : une recette avec une liste d’ingrédients, chacun avec son propre modèle `Ingredient`)
- Au moins un champ optionnel avec une valeur par défaut

Palier 3, Fonctionnalités avancées (obligatoire pour valider le projet)

Ce qu’on attend concrètement, avec des exemples d’URLs :

- Une route de **recherche ou de filtrage** via un ou plusieurs query params : `GET /recipes?category=dessert` renvoie uniquement les recettes de cette catégorie
- Une route avec **pagination** : `GET /recipes?limit=10&offset=20` renvoie 10 résultats à partir du 21ème
- Une route avec **tri** : `GET /recipes?sort_by=prep_time_minutes` change l’ordre des résultats renvoyés
- Une route de **statistiques**, qui calcule une donnée agrégée en Python à partir de vos données réelles, pas une valeur codée en dur : `GET /stats` pourrait renvoyer `{"total_recipes": 42, "average_prep_time": 27.5, "most_common_category": "dessert"}`

Palier 4, Qualité et robustesse (obligatoire pour valider le projet)

Ce qu’on attend concrètement :

- `HTTPException` propre sur au moins **4 cas d'erreur distincts**, avec des messages clairs. Par exemple :
 - `404` si une ressource n'existe pas
 - `404` si un identifiant lié (comme `author_id`) ne correspond à personne
 - `400` si une donnée est valide en soi mais incohérente avec le contexte
 - `403` ou `400` si une action est interdite dans votre logique métier
- `response_model` qui cache au moins un champ sensible, sur au moins **2 ressources différentes** (par exemple, cacher l'email d'un auteur dans la réponse publique)
- Un **journal de bord** (`JOURNAL.md`), rédigé par le groupe, qui explique en quelques phrases : les ressources choisies et pourquoi, les choix de validation les plus intéressants, et une difficulté rencontrée avec sa solution
- Un **journal de tests** (`TESTS.md`) qui liste, pour chaque route, au moins un cas de succès et un cas d'échec testés manuellement, avec le résultat obtenu. Exemple de ligne : `POST /recipes avec prep_time_minutes = -5 -> 422, message clair sur la contrainte ge`

Palier 5, Défi bonus (facultatif, pour les groupes en avance)

Choisissez au moins un de ces défis, ou proposez le vôtre en le faisant valider par moi avant de le commencer :

- Une route qui **duplique une ressource existante** avec un nouvel identifiant, en copiant en profondeur ses sous-objets imbriqués
- Un `model_validator` qui vérifie une cohérence **entre deux ressources différentes** (par exemple : une réservation ne peut pas être créée si l'événement affiche déjà complet)
- Une route d'**export** qui renvoie l'ensemble d'une ressource dans un format texte simple (CSV basique, généré en Python, sans librairie externe)
- Une route de **recherche globale** qui interroge plusieurs ressources en même temps et renvoie les résultats combinés

Le test croisé (dernière heure de la séance)

Environ une heure avant la fin, vous allez échanger l'URL de votre dépôt GitHub avec un autre groupe (je désignerai les binômes d'échange). Chaque groupe doit alors :

1. **Cloner le dépôt** de l'autre groupe (`git clone`, l'avantage d'un dépôt public), installer les dépendances et lancer leur API
2. Essayer de la faire échouer : données limites, types incorrects, identifiants inexistant, incohérences métier
3. Noter au moins 3 comportements suspects ou bugs trouvés, dans un fichier `REVIEW.md`
4. **Ouvrir une Pull Request** sur le dépôt de l'autre groupe avec ce fichier `REVIEW.md`, ou à défaut le déposer directement si la Pull Request pose problème techniquement

Ce test croisé fait partie de la note, sur deux aspects : la qualité de votre revue de l'API d'un autre groupe, et la robustesse de la vôtre face à la revue reçue.

Déroulé indicatif de la séance

Horaire	Étape
0h00, 0h20	Formation des groupes, création du dépôt GitHub, choix du thème
0h20, 1h00	Conception : lister vos ressources, leurs champs, leurs relations, avant de coder
1h00, 2h30	Développement des paliers 1 et 2, commits réguliers
2h30, 3h00	Développement du palier 3, puis 4 selon votre avancée
3h00, 3h45	Test croisé : clonage, revue, Pull Request sur le dépôt d'un autre groupe
3h45, 4h00	Corrections finales, préparation d'une mini démo (2 à 3 minutes)

Ne sautez pas la phase de conception. 40 minutes passées à lister vos ressources et leurs champs sur papier ou dans un document partagé vous feront gagner bien plus de temps ensuite, surtout avec 5 ressources à gérer.

Répartition suggérée des rôles

À titre indicatif seulement, à adapter librement par votre groupe :

- Un référent modèles et validation (Pydantic, paliers 1 et 2)
- Un référent fonctionnalités avancées (recherche, pagination, statistiques, palier 3)
- Un référent qualité, tests et documentation (palier 4, journal de bord, journal de tests, gestion du dépôt GitHub)

Ces rôles aident à s'organiser, mais tout le monde code, commit, et doit connaître l'ensemble du projet pour la présentation finale.

Livrable

Tout doit se trouver sur votre dépôt GitHub public à la fin de la séance :

- Le code source complet
- Le fichier `README.md` expliquant le thème choisi, les ressources de l'API, comment lancer le projet
- Le fichier `JOURNAL.md` (palier 4)
- Le fichier `TESTS.md` (palier 4)
- Le fichier `REVIEW.md` reçu du groupe qui a testé votre API
- Un historique de commits qui reflète une progression tout au long de la séance

Grille d'évaluation

Critère	Poids
Palier 1 et 2 (fondations et validation)	25%
Palier 3 (fonctionnalités avancées)	20%
Palier 4 (robustesse, journal, tests)	20%
Qualité du dépôt GitHub (structure, historique de commits, README)	15%
Qualité de la revue croisée effectuée et réaction à celle reçue	10%
Présentation finale du groupe	10%
Défi bonus du palier 5	jusqu'à 5% supplémentaires

Conseils

- Mieux vaut atteindre solidement le palier 3 que bâcler jusqu'au palier 5
- Testez au fur et à mesure, pas seulement à la fin
- Committez souvent, un historique de commits détaillé raconte l'histoire de votre projet mieux qu'un long README
- Si vous bloquez plus de 5 minutes en groupe sans avancer, appelez-moi

En cas de blocage

J'appellerai chaque groupe rapidement au moment de la validation du thème, mais je circule aussi tout le reste de la séance. N'hésitez pas à m'appeler, y compris pour un problème Git.